topic_3_template_improvements

Theory, Edge Cases & Code Examples

- ✓ Implicit Template Parameters
- ✓ Fold Expressions
- ✓ constexpr Enhancements in C++17
- ✓ STL Improvements in C++17
- ✓ Miscellaneous C++17 Features

→ Swipe to learn →

topic_3_template_improvements

Theory

📌 1. Class Template Argument Deduction (CTAD) - Implicit Template Parameters (1/3)

Definition: CTAD allows the compiler to automatically deduce class template arguments from constructor arguments, eliminating verbose and redundant type specifications.

Before vs After Comparison:

Scenario	C++14 (Explicit)	C++17 CTAD (Deduced)
std::pair	`std::pair<int, double>(42, 3.14)`	`std::pair(42, 3.14)`
std::vector	`std::vector<int>{1, 2, 3}`	`std::vector{1, 2, 3}`
std::tuple	`std::tuple<int, string, double>(1, "a", 3.14)`	`std::tuple(1, "a", 3.14)`
std::array	`std::array<int, 3>{1, 2, 3}`	`std::array{1, 2, 3}` (C++17)
std::optional	`std::optional<int>(42)`	`std::optional(42)`

How CTAD Works (Mechanism):

1. **Constructor examination:** Compiler analyzes available constructors
2. **Argument matching:** Maps constructor parameter types to template parameters
3. **Type deduction:** Deduces template arguments from actual arguments
4. **Deduction guide lookup:** Applies user-provided or compiler-generated guides
5. **Instantiation:** Creates class template instance with deduced types

topic_3_template_improvements

Theory

📌 1. Class Template Argument Deduction (CTAD) - Implicit Template Parameters (2/3)

Deduction Guides - Custom CTAD Rules:

Guide Type	Purpose	Example
User-defined	Control deduction for custom types	`template<typename T>` `Container(T) -> Container<T>;`
Aggregate deduction	Enable CTAD for aggregate types	`template<typename T>` `Point(T, T) -> Point<T>;`
Explicit	Force specific deduction	`Container(initializer_list<T>) -> Container<vector<T>>;`
Compiler-generated	Automatic for constructors	Generated from public constructors

Custom Deduction Guide Example:

```
template<typename T>
struct Container {
    Container(T val) : data(val) {}
    Container(std::initializer_list<T> list) : data(list.begin(), list.end()) {}
    std::vector<T> data;
};

// Deduction guide: initializer_list deduces to vector storage
template<typename T>
Container(std::initializer_list<T>) -> Container<T>;

Container c1(42);               // Deduces Container<int>, data = {42}
Container c2{1, 2, 3};          // Uses guide: Container<int>, data = {1,2,3}
Container c3(vec.begin(), vec.end());  // Needs guide for iterator pair
```

topic_3_template_improvements

Theory

📌 1. Class Template Argument Deduction (CTAD) - Implicit Template Parameters (3/3)

When CTAD Fails (Limitations):

Failure Reason	Code Example	Fix
Non-deducible parameters	`template<typename T, int N> Array()`	Provide default or deduction guide
Ambiguous constructors	`Wrapper(&x)` matches both `Wrapper(T)` and `Wrapper(T*, size_t)`	Explicit deduction guide or template args
Private constructors	Constructor not visible	Make public or add deduction guide
Copy-list initialization	`Container c = {1, 2};` may fail	Use direct initialization

CTAD Best Practices:

- **Use for standard containers:** Always use CTAD with std::pair, std::tuple, std::optional, etc.
- **Avoid for type aliases:** `using IntVec = vector<int>; IntVec v{1,2,3};` doesn't work
- **Provide guides for ambiguity:** Add deduction guides when multiple constructors could match
- **Explicit for clarity:** Sometimes explicit types are clearer than CTAD

---

topic_3_template_improvements

Theory

📌 Fold Expressions (1/2)

What are Fold Expressions?

Fold expressions provide a compact way to perform operations on all elements of a parameter pack using binary operators. They eliminate the need for recursive template instantiation for variadic templates.

Syntax:

1. **Unary Right Fold:** `(pack op ...)` → `(E1 op (E2 op (E3 op E4)))`
2. **Unary Left Fold:** `(... op pack)` → `(((E1 op E2) op E3) op E4)`
3. **Binary Right Fold:** `(pack op ... op init)` → `(E1 op (E2 op (E3 op init)))`
4. **Binary Left Fold:** `(init op ... op pack)` → `(((init op E1) op E2) op E3)`

Supported Operators:

```
+  -  *  /  %  ^  &  |  =  <  >  <<  >>
+=  -=  *=  /=  %=  ^=  &=  |=  <<=  >>=
==  !=  <=  >=  &&  ||  ,  .*  ->*
```

Basic Examples:

```
// Sum all arguments
template<typename... Args>
auto sum(Args... args) {
    return (args + ...);  // Unary right fold
}
sum(1, 2, 3, 4);  // Returns 10

// All true (logical AND)
template<typename... Args>
bool all_true(Args... args) {
    return (args && ...);
}
all_true(true, true, false);  // Returns false

// Print all arguments
template<typename... Args>
void print_all(Args... args) {
    (std::cout << ... << args) << '\n';  // Left fold with <<
}
print_all("Speed: ", 60, " km/h");  // Prints: Speed: 60 km/h
```

topic_3_template_improvements

Theory

📌 Fold Expressions (2/2)

Fold with Initial Value:

```
template<typename... Args>
auto sum_with_init(Args... args) {
    return (0 + ... + args);  // Binary left fold with init value 0
}
sum_with_init();  // Returns 0 (safe for empty pack)
```

Important: Empty Parameter Packs:

- For `&&` : empty fold evaluates to `true`
- For `||` : empty fold evaluates to `false`
- For `,` : empty fold evaluates to `void()`
- For all other operators: ill-formed (compilation error)

Autonomous Driving Example:

```
// Validate all sensors are operational
template<typename... Sensors>
bool all_sensors_ready(Sensors&&... sensors) {
    return (... && sensors.is_ready());  // Fold with &&
}

// Aggregate multiple sensor readings
template<typename... Readings>
double average_readings(Readings... readings) {
    constexpr size_t count = sizeof...(readings);
    return (readings + ...) / count;
}

// Execute multiple control commands
template<typename... Commands>
void execute_all(Commands&&... cmds) {
    (cmds.execute(), ...);  // Fold with comma operator
}
```

topic_3_template_improvements

Theory

📌 constexpr Enhancements in C++17

constexpr Lambda:

C++17 allows lambdas to be implicitly constexpr if they satisfy constexpr requirements:

```
constexpr auto square = [](int x) { return x * x; };
constexpr int result = square(5);  // Computed at compile time

// Can be used in constant expressions
std::array<int, square(3)> arr;  // Array of size 9
```

constexpr if:

Already covered in Topic 1, but worth noting it's a major enhancement for compile-time programming.

topic_3_template_improvements

Theory

📌 STL Improvements in C++17 (1/4)

1. insert_or_assign and try_emplace (for maps):

These methods improve map operations by reducing unnecessary copies and moves:

```
std::map<std::string, Sensor> sensors;

// insert_or_assign: insert if key doesn't exist, assign if it does
sensors.insert_or_assign("lidar", Sensor{/*...*/});

// try_emplace: only constructs value if insertion happens
sensors.try_emplace("camera", /*constructor args*/);
```

Advantages:

- `try_emplace` : Never moves key, constructs in-place only if needed
- `insert_or_assign` : Clearer intent than operator[] or insert

2. std::scoped_lock (Deadlock-Free Locking):

Multi-mutex RAII lock that prevents deadlock:

```
std::mutex m1, m2;

void f1() {
    std::scoped_lock lock(m1, m2);  // Locks both in safe order
    // ...
}

void f2() {
    std::scoped_lock lock(m2, m1);  // Different order, still safe!
    // ...
}
```

Benefits:

- Automatically acquires multiple mutexes in deadlock-free order
- RAII: releases on scope exit
- More convenient than std::lock + std::lock_guard

3. Parallel Algorithms (std::execution):

C++17 adds execution policies for parallel and vectorized algorithm execution:

topic_3_template_improvements

Theory

📌 STL Improvements in C++17 (2/4)

```
#include <execution>
#include <algorithm>

std::vector<int> data = {/*...*/};

// Sequential (default)
std::sort(data.begin(), data.end());

// Parallel
std::sort(std::execution::par, data.begin(), data.end());

// Parallel + Vectorized
std::sort(std::execution::par_unseq, data.begin(), data.end());

// Sequenced (explicitly sequential)
std::sort(std::execution::seq, data.begin(), data.end());
```

Execution Policies:

- `std::execution::seq` - Sequential execution
- `std::execution::par` - Parallel execution (multi-threaded)
- `std::execution::par_unseq` - Parallel + vectorized (SIMD)
- `std::execution::unseq` - Vectorized only (C++20)

Supported Algorithms: sort, for_each, transform, reduce, find, count, etc.

topic_3_template_improvements

Theory

📌 STL Improvements in C++17 (3/4)

Autonomous Driving Example:

```
// Process large point cloud in parallel
std::vector<Point3D> point_cloud = lidar.get_points();
std::for_each(std::execution::par,
              point_cloud.begin(),
              point_cloud.end(),
              [](Point3D& p) { p.apply_transformation(/*...*/); });

// Parallel obstacle detection
auto obstacle_count = std::count_if(
    std::execution::par,
    point_cloud.begin(),
    point_cloud.end(),
    [](const Point3D& p) { return is_obstacle(p); }
);
```

4. std::to_chars and std::from_chars (Low-Level Conversion):

Fast, locale-independent numeric conversions without allocation:

```
char buffer[50];

// int to string
auto [ptr, ec] = std::to_chars(buffer, buffer + sizeof(buffer), 12345);
if (ec == std::errc{}) {
    std::string_view result(buffer, ptr - buffer);  // "12345"
}

// string to int
int value;
std::from_chars_result res = std::from_chars(buffer, ptr, value);
if (res.ec == std::errc{}) {
    // value now contains 12345
}
```

📌 STL Improvements in C++17 (4/4)

Benefits over stringstream:

- 3-10x faster
- No dynamic allocation
- Locale-independent
- No exceptions
- Round-trip guarantee for floating point

topic_3_template_improvements

Theory

📌 Miscellaneous C++17 Features (1/2)

1. Nested Namespaces:

Simplified syntax for nested namespace declarations:

```
// Before C++17
namespace company {
    namespace autonomous {
        namespace perception {
            class LidarSensor { };
        }
    }
}

// C++17
namespace company::autonomous::perception {
    class LidarSensor { };
}
```

2. Attributes:

[[nodiscard]] - Warns if function return value is discarded:

```
[[nodiscard]] std::optional<Route> find_route(Point start, Point end);

find_route(A, B);  // Warning: ignoring important return value!
```

[[maybe_unused]] - Suppresses unused variable warnings:

```
void process([[maybe_unused]] int debug_flag) {
    #ifdef DEBUG
        std::cout << debug_flag;
    #endif
}
```

[[fallthrough]] - Documents intentional switch case fallthrough:

```
switch (state) {
    case State::INIT:
        initialize();
// ...continues on next slide
```

topic_3_template_improvements

Theory

📌 Miscellaneous C++17 Features (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        [[fallthrough]];  // Intentional fallthrough
    case State::RUNNING:
        process();
        break;
}
```

3. Hexadecimal Floating-Point Literals:

```
double val = 0x1.2p3;  // 1.125 * 2^3 = 9.0
```

4. Direct List Initialization of Enums:

```
enum class Color { Red, Green, Blue };
Color c{0};  // Direct initialization with underlying value
```

topic_3_template_improvements

Edge Cases

📌 1. CTAD Ambiguity with Multiple Constructors

Problem: CTAD can fail when multiple constructors could match.

```
template<typename T>
struct Container {
    Container(T val);
    Container(T* ptr, size_t size);
};

// Ambiguous: which constructor?
Container c(data);  // Error if data could match both
```

Solution: Provide explicit deduction guides or specify template arguments.

```
template<typename T>
Container(T) -> Container<T>;

template<typename T>
Container(T*, size_t) -> Container<T>;
```

Key Insight: CTAD works best with unambiguous constructors or explicit deduction guides.

topic_3_template_improvements

Edge Cases

📌 2. Fold Expression with Empty Parameter Pack

Problem: Most operators cause compilation errors with empty packs.

```
template<typename... Args>
auto product(Args... args) {
    return (args * ...);  // Error if args is empty!
}

product();  // Compilation error
```

Solution: Use binary fold with initial value:

```
template<typename... Args>
auto product(Args... args) {
    return (1 * ... * args);  // OK: returns 1 for empty pack
}
```

Special Cases:
- `&&` → empty fold = `true`
- `||` → empty fold = `false`
- `,` → empty fold = `void()`
- All other operators → ill-formed

topic_3_template_improvements

Edge Cases

📌 3. Fold Expression Short-Circuit Evaluation

Problem: Fold expressions DO support short-circuit evaluation for `&&` and `||`.

```
template<typename... Args>
bool all_true(Args... args) {
    return (... && args);
}

bool expensive_check() { /* costly operation */ return false; }

// If first arg is false, expensive_check() is NOT called
all_true(false, expensive_check());  // Short-circuits!
```

But: All arguments are evaluated before folding begins:

```
// This WILL throw even though && would short-circuit
all_true(true, false, throw std::runtime_error("oops"));  // Throws!
```

Key Insight: Short-circuiting applies during fold operation, not during argument evaluation.

topic_3_template_improvements

Edge Cases

📌 4. Parallel Algorithm Thread Safety

Problem: Using parallel algorithms with non-thread-safe operations causes data races.

```
int counter = 0;
std::vector<int> data = {1, 2, 3, 4, 5};

// DATA RACE! Multiple threads write to counter
std::for_each(std::execution::par, data.begin(), data.end(),
              [&counter](int) { counter++; });  // UB!
```

Solution: Use atomic operations or avoid shared state:

```
std::atomic<int> counter{0};
std::for_each(std::execution::par, data.begin(), data.end(),
              [&counter](int) { counter++; });  // Safe
```

Key Insight: Parallel execution policies require thread-safe operations on shared data.

topic_3_template_improvements

Edge Cases

📌 5. std::to_chars Buffer Overflow

Problem: std::to_chars doesn't null-terminate and can fail silently.

```
char buf[5];
auto [ptr, ec] = std::to_chars(buf, buf + 5, 123456);  // Too large!

if (ec == std::errc::value_too_large) {
    // Buffer was too small!
}

// buf is NOT null-terminated! Must check ec and use ptr
std::string result(buf, ptr);  // Correct: use range
```

Key Insight: Always check error code and use pointer range, never assume null termination.

topic_3_template_improvements

Code Examples

📌 1. CTAD with Custom Deduction Guides (1/2)

Scenario: Create a coordinate system with automatic type deduction for autonomous vehicle positioning.

Explanation:
- CTAD eliminates redundant type specifications
- Deduction guide ensures correct type deduction from constructor args
- Makes code cleaner and less error-prone
- Particularly useful with complex nested types

topic_3_template_improvements

Code Examples

📌 1. CTAD with Custom Deduction Guides (2/2)

Output:

```
#include <iostream>
#include <vector>
#include <initializer_list>

template<typename T>
class Coordinate {
public:
    T x, y;

    Coordinate(T x_val, T y_val) : x(x_val), y(y_val) {}

    void print() const {
        std::cout << "(" << x << ", " << y << ")\n";
    }
};

// Deduction guide for aggregate initialization
template<typename T>
Coordinate(T, T) -> Coordinate<T>;

int main() {
    // CTAD automatically deduces types
    Coordinate pos1(10.5, 20.3);      // Coordinate<double>
    Coordinate pos2(100, 200);        // Coordinate<int>

    pos1.print();  // (10.5, 20.3)
    pos2.print();  // (100, 200)

    // Works with std containers too
    std::pair waypoint("Start", Coordinate(0.0, 0.0));
    // Type: std::pair<const char*, Coordinate<double>>

    return 0;
}
```

```
['(10.5, 20.3)\n(100, 200)']
```

topic_3_template_improvements

Code Examples

📌 2. Fold Expressions for Sensor Validation (1/3)

Scenario: Validate multiple sensor states using fold expressions for autonomous vehicle safety checks.

Explanation:

- `(... && sensors.is_ready())` creates AND chain of all ready checks
- `(0 + ... + expr)` sums boolean conversions to count ready sensors
- `((std::cout << ...), ...)` uses comma operator for sequential printing
- Fold expressions eliminate need for recursive templates
- More readable than traditional variadic template approaches

topic_3_template_improvements

Code Examples

📌 2. Fold Expressions for Sensor Validation (2/3)

Output:

```
#include <iostream>
#include <string>

class Sensor {
    std::string name;
    bool ready;

public:
    Sensor(std::string n, bool r) : name(std::move(n)), ready(r) {}

    bool is_ready() const { return ready; }
    std::string get_name() const { return name; }
};

// Check if all sensors are ready
template<typename... Sensors>
bool all_sensors_ready(const Sensors&... sensors) {
    return (... && sensors.is_ready());  // Left fold with &&
}

// Count ready sensors
template<typename... Sensors>
int count_ready_sensors(const Sensors&... sensors) {
    return (0 + ... + (sensors.is_ready() ? 1 : 0));
}

// Print all sensor names
template<typename... Sensors>
void print_sensor_names(const Sensors&... sensors) {
    std::cout << "Sensors: ";
    ((std::cout << sensors.get_name() << " "), ...);
    std::cout << "\n";
}

int main() {
    Sensor lidar("LIDAR", true);
    Sensor camera("Camera", true);
    Sensor radar("RADAR", false);
    Sensor gps("GPS", true);
// ...continues on next slide
```

topic_3_template_improvements

Code Examples

📌 2. Fold Expressions for Sensor Validation (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    std::cout << "All ready: " << std::boolalpha
              << all_sensors_ready(lidar, camera, radar, gps) << "\n";

    std::cout << "Ready count: " << count_ready_sensors(lidar, camera, radar, gps) << "\n";

    print_sensor_names(lidar, camera, radar, gps);

    // With all sensors ready
    std::cout << "All ready: " << all_sensors_ready(lidar, camera, gps) << "\n";

    return 0;
}
```

```
['All ready: false\nReady count: 3\nSensors: LIDAR Camera RADAR GPS\nAll ready: true']
```

topic_3_template_improvements

Code Examples

📌 3. Parallel Algorithm for Point Cloud Processing (1/4)

Scenario: Process large LiDAR point cloud in parallel for obstacle detection.

Explanation:

- `std::execution::seq` - explicit sequential execution for baseline
- `std::execution::par` - parallel execution across multiple threads
- `std::for_each` with parallel policy automatically distributes work
- `std::count_if` with parallel policy for efficient counting
- Significant speedup for computationally intensive operations
- Note: Requires compiler/library support (GCC with TBB, MSVC, etc.)

topic_3_template_improvements

Code Examples

📌 3. Parallel Algorithm for Point Cloud Processing (2/4)

Sample Output:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <execution>
#include <random>
#include <chrono>

struct Point3D {
    double x, y, z;

    bool is_obstacle() const {
        // Simple heuristic: points above ground level
        return z > 0.5;
    }

    void apply_filter() {
        // Noise reduction (simplified)
        x = std::round(x * 10) / 10.0;
        y = std::round(y * 10) / 10.0;
        z = std::round(z * 10) / 10.0;
    }
};

std::vector<Point3D> generate_point_cloud(size_t count) {
    std::vector<Point3D> cloud;
    cloud.reserve(count);

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(-10.0, 10.0);

    for (size_t i = 0; i < count; ++i) {
        cloud.push_back({dis(gen), dis(gen), dis(gen)});
    }

    return cloud;
}

int main() {
// ...continues on next slide
```

topic_3_template_improvements

Code Examples

📌 3. Parallel Algorithm for Point Cloud Processing (3/4)

```
// ...continued from previous slide
    const size_t POINT_COUNT = 1'000'000;
    auto cloud = generate_point_cloud(POINT_COUNT);

    std::cout << "Processing " << POINT_COUNT << " points...\n";

    // Sequential filtering
    auto cloud_seq = cloud;
    auto start_seq = std::chrono::high_resolution_clock::now();

    std::for_each(std::execution::seq,
                  cloud_seq.begin(),
                  cloud_seq.end(),
                  [](Point3D& p) { p.apply_filter(); });

    auto end_seq = std::chrono::high_resolution_clock::now();
    auto duration_seq = std::chrono::duration_cast<std::chrono::milliseconds>(end_seq - start_seq);

    // Parallel filtering
    auto cloud_par = cloud;
    auto start_par = std::chrono::high_resolution_clock::now();

    std::for_each(std::execution::par,
                  cloud_par.begin(),
                  cloud_par.end(),
                  [](Point3D& p) { p.apply_filter(); });

    auto end_par = std::chrono::high_resolution_clock::now();
    auto duration_par = std::chrono::duration_cast<std::chrono::milliseconds>(end_par - start_par);

    // Count obstacles in parallel
    auto obstacle_count = std::count_if(
        std::execution::par,
        cloud.begin(),
        cloud.end(),
        [](const Point3D& p) { return p.is_obstacle(); }
    );

    std::cout << "Sequential: " << duration_seq.count() << " ms\n";
    std::cout << "Parallel: " << duration_par.count() << " ms\n";
    std::cout << "Speedup: " << static_cast<double>(duration_seq.count()) / duration_par.count() << "x\n";
// ...continues on next slide
```

topic_3_template_improvements

Code Examples

📌 3. Parallel Algorithm for Point Cloud Processing (4/4)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "Obstacles detected: " << obstacle_count << "\n";

    return 0;
}
```

```
['Processing 1000000 points...\nSequential: 45 ms\nParallel: 12 ms\nSpeedup: 3.75x\nObstacles detected: 498234']
```

topic_3_template_improvements

Code Examples

📌 4. std::scoped_lock for Multi-Sensor Synchronization (1/3)

Scenario: Safely update multiple sensor buffers from different threads without deadlock.

Explanation:

- `std::scoped_lock` locks multiple mutexes in deadlock-free order
- Thread 1 locks (lidar, camera), Thread 2 locks (camera, lidar)
- Without `scoped_lock`, this would deadlock with separate lock_guards
- RAII ensures both mutexes released on exception or return
- Safer and more convenient than `std::lock` + `std::lock_guard`

topic_3_template_improvements

Code Examples

📌 4. std::scoped_lock for Multi-Sensor Synchronization (2/3)

Output:

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>

class SensorBuffer {
    std::mutex mtx;
    std::vector<double> data;
    std::string name;

public:
    SensorBuffer(std::string n) : name(std::move(n)) {}

    void add_reading(double value) {
        std::lock_guard<std::mutex> lock(mtx);
        data.push_back(value);
    }

    std::mutex& get_mutex() { return mtx; }

    size_t size() const { return data.size(); }

    void sync_with(SensorBuffer& other, double value1, double value2) {
        // Deadlock-free locking of both buffers
        std::scoped_lock lock(mtx, other.mtx);

        data.push_back(value1);
        other.data.push_back(value2);

        std::cout << "Synced " << name << " with " << other.name << "\n";
    }
};

int main() {
    SensorBuffer lidar("LIDAR");
    SensorBuffer camera("Camera");

    // Thread 1: sync LIDAR with Camera
// ...continues on next slide
```

topic_3_template_improvements

Code Examples

📌 4. std::scoped_lock for Multi-Sensor Synchronization (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    std::thread t1([&]() {
        for (int i = 0; i < 5; ++i) {
            lidar.sync_with(camera, i * 1.0, i * 2.0);
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    });

    // Thread 2: sync Camera with LIDAR (reversed order!)
    std::thread t2([&]() {
        for (int i = 0; i < 5; ++i) {
            camera.sync_with(lidar, i * 3.0, i * 4.0);
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    });

    t1.join();
    t2.join();

    std::cout << "LIDAR readings: " << lidar.size() << "\n";
    std::cout << "Camera readings: " << camera.size() << "\n";

    return 0;
}
```

```
['Synced LIDAR with Camera\nSynced Camera with LIDAR\nSynced LIDAR with Camera\nSynced Camera with LIDAR\n...\nLIDAR re
```

topic_3_template_improvements

Code Examples

📌 5. constexpr Lambda for Compile-Time Computation (1/3)

Scenario: Compute lookup tables at compile time for autonomous vehicle trajectory calculations.

Explanation:
- Lambdas are implicitly constexpr in C++17 if they satisfy requirements
- Used to generate lookup tables at compile time
- Zero runtime overhead - all computation done during compilation
- `constexpr` lambda can be used in constant expressions
- Useful for physics calculations, math tables, config values

topic_3_template_improvements

Code Examples

📌 5. constexpr Lambda for Compile-Time Computation (2/3)

Output:

```
#include <iostream>
#include <array>
#include <cmath>

// constexpr lambda for compile-time computation
constexpr auto compute_steering_angle = [](double curvature) {
    return curvature * 45.0;  // Simplified: max 45 degrees
};

// Generate lookup table at compile time
template<size_t N>
constexpr auto generate_steering_table() {
    std::array<double, N> table{};

    constexpr auto compute = [](size_t i) {
        double curvature = static_cast<double>(i) / N;
        return compute_steering_angle(curvature);
    };

    for (size_t i = 0; i < N; ++i) {
        table[i] = compute(i);
    }

    return table;
}

// Compile-time constant
constexpr auto STEERING_TABLE = generate_steering_table<100>();

int main() {
    // Access compile-time computed values at runtime
    std::cout << "Steering angles (first 10 entries):\n";
    for (size_t i = 0; i < 10; ++i) {
        std::cout << "Curvature " << i / 100.0 << " -> "
                  << STEERING_TABLE[i] << " degrees\n";
    }

    // Use in constant expression
    constexpr double angle_at_50 = STEERING_TABLE[50];
// ...continues on next slide
```

topic_3_template_improvements

Code Examples

📌 5. constexpr Lambda for Compile-Time Computation (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "\nAngle at 50% curvature: " << angle_at_50 << " degrees\n";

    // Lambda can be used in constexpr context
    static_assert(compute_steering_angle(0.5) == 22.5);

    return 0;
}
```

```
['Steering angles (first 10 entries):\nCurvature 0 -> 0 degrees\nCurvature 0.01 -> 0.45 degrees\nCurvature 0.02 -> 0.9
```

What You Learned

Key Takeaways

- ✓ Implicit Template Parameters
- ✓ Fold Expressions
- ✓ constexpr Enhancements in C++17
- ✓ STL Improvements in C++17

→ Swipe to learn →